Each section states a decision, the formal result it rests on, the citation, and the measurement from this project that confirms or contradicts it. Where a decision is only partly defensible, that is said plainly rather than dressed up.

Companion to `docs/METHODOLOGY.md`, which covers the experimental design (paired comparison, factorial vs OFAT, self-play). This file covers the decisions made after that document was written.

Decision. Every behavioural statistic is computed over games passing `engine/quality.js`. That is 1,865 of 14,794 records — 13%.

The formal point. This is not a sample-size trade-off, it is a **target-population** question. An estimator is unbiased *for the population it samples from*. The quantity we want is "how do humans play Champions"; the raw store estimates "how do humans-and-bots-and-forfeiters play". Adding contaminated records does not reduce the variance of the estimate we want — it estimates a different parameter with more precision. Precision about the wrong quantity is not an improvement.

The circularity is the sharper problem. Self-play exists to produce games that resemble *human* play. Measuring our bots against a corpus that is 63% other people's bots and calling the difference "realism" evaluates the model against a population it was never meant to imitate.

Measured, and it cuts both ways. The bot games are *not* weak:

	clean humans	ladder bots	our self-play
super effective	21.4%	**24.2%**	11.2%
moves that failed	2.5%	**2.8%**	8.6%
turns per game	8.4	6.2	10.8

So contamination shifted the target by ~3 points while our own gaps are 10–18. **Direction of every conclusion survived; precision did not.** Reported here because the honest reading is "this mattered less than feared", not "this vindicated the filter".

The residual worry, stated. 1,865 games is thin, and replays are uploaded *voluntarily* — a self-selected sample, so even the clean subset is not a random draw from the ladder. This is why anything about team composition is taken from Smogon (1,163,315 battles, computed server-side over every team played) rather than from our store. Our store is used only for what Smogon does not publish: per-turn behaviour.

Decision. In `set_space.js`, a move Smogon reports on ≥50% of sets is treated as fixed when deciding which slots a build experiment should vary. It is **not** force-included during set generation — see the limit below, which is the more instructive half of this section.

The formal result. Under 0–1 loss on a discrete outcome, the Bayes-optimal decision is the **mode of the conditional distribution**, `argmax_a p(Y = a | X)` — not a draw from that distribution (Stephens, *Bayes decision rule for prediction*). Sampling in proportion to the probabilities is **probability matching**, and it is strictly suboptimal for accuracy; the Bayes rule pushes decisions toward 0 or 1 rather than tracking the base rate (Bayesian optimal treatment regimes, arXiv 1809.06679).

Specialised to one binary slot: always-including a move that truly sits on a fraction `p` of sets reproduces reality on `p` of sets. Sampling it with probability `p` reproduces reality on `p² + (1−p)²`. The difference is

```
p − (p² + (1−p)²) = (2p − 1)(1 − p)
```

positive exactly when `p > ½`. **The crossover is arithmetic, not a tuned threshold** — which is what makes it admissible under S12/S13.

THE HONEST LIMIT — AND IT IS LARGER THAN THIS SECTION FIRST CLAIMED.

This was implemented, measured, and reverted the same hour. Forcing every move above 50% put all of them on 100% of generated sets and drove Kingambit's Low Kick from 47.8% to **zero**, Swords Dance to **zero**. Mean deviation from Smogon went from 2.6 points under to 17.5 points over. The set space collapsed to one build per species.

The error was applying the right theorem to the wrong objective. Two different goals are in play and they have *opposite* optima:

goal	optimal rule
guess ONE opponent's set as accurately as possible	**take the mode** (Bayes, 0-1 loss)
generate a CORPUS whose sets are distributed like reality	**sample from the distribution**

Probability matching is suboptimal for per-instance accuracy and *correct* for distributional fidelity — a generated corpus is closest to the truth in distribution when it is drawn from the truth. Set generation exists to populate a corpus, so it wants the second rule. XATU, which predicts a specific opponent's set from partial revelation, wants the first.

So the 50% rule stands where it was derived — `set_space.js`, choosing which slots a build experiment should hold fixed — and does **not** govern set generation. The earlier text in this section implied otherwise and was wrong.

The real defect, correctly sized. Our locked moves appear 2.6 points *less* often than Smogon says (Farigiraf 5.8, Kingambit 5.5, Sinistcha 0.3) — a small, systematic under-production, not a missing rule. Its cause is the sequential weighted draw: once common moves are taken, the four-slot constraint forces rare ones onto the set. The principled fix is to sample each move independently at its Smogon *set-rate* and repair to exactly four, which targets the marginals directly. Open, and deliberately not fixed by forcing.

Decision. Report, and refuse to chase, the fraction of real sets containing a move Smogon does not list individually.

The formal frame. This is the **unseen-species problem**, posed by Corbet to Fisher in the 1940s and solved by Good and Turing in 1953: estimate the total probability of outcomes absent from the sample (Good–Turing overview; Generalized Good–Turing, *JASA* 118(543)). The quantity is the **missing mass**, and estimating it is a standard, well-posed problem with known minimax risk (arXiv 1705.05006).

Our situation is the easy case. Smogon *publishes* the missing mass directly as the "Other" bucket, so no estimator is needed — the tail probability is given. Converting per-slot mass to per-set probability uses only the fact that a set holds four moves:

```
P(set contains an unlisted move) = 1 - (1 - other/400)^4  ≈ 17%
```

Median 17% across 283 species, worst 19%, Kingambit 3%.

Why this framing matters practically. Missing mass is a property of the *sampling process*, not a defect in our pipeline. No amount of work on our side recovers it, because the information was discarded before publication. Naming it correctly is what licenses the decision to stop — and this framing is being used the same way in ML deployment-coverage work (Blind-Spot Mass, arXiv 2604.05057).

Caveat. Independence across the four slots is assumed and is certainly false — movesets are correlated. The figure is therefore approximate, and should be read as "about one set in six", not as a calibrated number.

Decision. In `build_lab`, the Pokémon whose build is being measured selects uniformly over its legal moves. Opponents keep the usage-weighted behaviour clone.

The problem it solves. Usage priors are renormalised over whichever four moves a build carries, so the *amount of measurement each arm receives depends on the arm*. Measured on Garchomp: the tested slot takes 29.4% of the pilot's decisions as Earthquake and 25.2% as Stomping Tantrum — **0.86×**. A rare move gets less opportunity to demonstrate its effect, and rarity is the treatment under study.

This is treatment-correlated measurement intensity — a confound, not noise. It biases toward builds made of common moves, which is circular, since commonality is the hypothesis being tested rather than a fact to be assumed. Equal allocation across arms is the standard remedy: with equal variances, balanced allocation minimises the variance of pairwise contrasts, which is why balance is the default in comparative designs (Czitrom, *One-Factor-at-a-Time Versus Designed Experiments*).

THE FLAW, which is real and was pointed out immediately. Uniform allocation is unbiased across moves **only if the moves are exchangeable in time**. Setup moves are not: Swords Dance, Trick Room and Tailwind have value only when played *before* the attacks they enable. A uniform pilot clicks Swords Dance on the last turn, or twice, and a build containing it is penalised for reasons unrelated to the build.

So the change removes a bias in *how much* each move is measured and leaves a bias in *when*. That is an improvement — the first is proportional to the treatment, the second is a property of the pilot shared by all arms carrying setup — but it is not a solution, and `build_lab` currently understates setup builds.

The three options, ranked.

1. **Hand-code "setup first."** Rejected: asserted domain logic, and the list never ends.
2. **Condition the move prior on turn index**, measured from clean games. Derived rather than typed, and fixes most of the effect. Defensible as an interim.
3. **A board-conditioned policy.** The right answer. Clicking setup when the turn is free is not a rule about setup moves; it is what reading the board means. Everything above is scaffolding until it exists.

UPDATE (v3.7.0): option 3 now exists — and it does not fix this. `engine/magnemite.js` reads the board (§6), so the *opponents* in a `build_lab` run are board-conditioned. The **species under test is deliberately excluded**, because the argument at the top of this section still holds: equal airtime per arm is what makes the comparison unbiased, and a board-aware pilot allocates airtime by how good each move looks, which is the treatment under study.

So the two halves of this section have come apart. The bias in *how much* each move is measured is still removed, by the uniform pilot. The bias in *when* is still there, unchanged, because the pilot that would fix it is the one thing this design cannot use. Choosing between equal airtime and sensible timing is a real design question and it is open; it is not a patch, and it should not be recorded as solved.

Decision. Per-species build structure ("freedom") is computed as `(400 − sum of the top four moves) / 100` rather than from a usage threshold.

Why it is not a modelling choice. Smogon's move percentages are shares of *sets*, and every set holds exactly four moves, so the listed percentages plus "Other" sum to 400 by construction. The quantity is therefore a **compositional constraint**, not an estimate: the residual after the four most common moves is, by arithmetic, the expected number of slots differing from the standard build.

Garchomp 0.71, Farigiraf 1.47, Kingambit 0.59. Across 259 species with 2,000+ teams the four most common moves account for 68% of all move slots played.

This replaced a hand-written `LOCK_AT = 85`, which was both invented and wrong — it classified Garchomp's Earthquake (76.9%) as a free choice.

Decision. The board-aware policy's weights are **estimated from human clicks** by conditional logit, rather than written as a scoring function and tuned.

The formal frame. At each decision a player faced a set of alternatives — every (move, target) pair legal that turn — and picked one. That is the canonical **discrete choice** setting, and the conditional logit model is McFadden's (*Conditional Logit Analysis of Qualitative Choice Behavior*, in Zarembka (ed.), Frontiers in Econometrics, 1974), the work the 2000 Nobel cited. Each alternative `j` carries attributes `x_j`; the model is

```
P(pick j) = exp(w . x_j) / sum_k exp(w . x_k)
```

and `w` is chosen to maximise the log-likelihood of what people actually did. Crucially the attributes belong to the **alternatives**, not to the chooser, which is exactly our situation: "how effective is this move against that specific foe" is a property of the option.

Why estimated and not written down. The obvious alternative is `score = power × effectiveness` with coefficients tuned until the realism report looks right. That has as many free parameters as it has terms, every one of them asserted, and tuning them against the number being reported is circular — the report stops being evidence the moment it becomes the objective. Here the realism report is never consulted during fitting and is held back as the out-of-sample check. It is the same discipline as §5: prefer the quantity that arithmetic or data fixes over the one a person picks.

Measured. Held out **by game** — decisions within a game share teams, players and board, so splitting by decision leaks across the split:

model	logL/decision	top-1
uniform over candidates	−1.7627	24.1%
behaviour clone alone	−1.9302	27.1%
board-aware fit	−1.6006	33.6%

A result that was not expected and is worth keeping. The behaviour clone alone is a *worse* probabilistic model of human choice than picking uniformly at random, and the fit assigns it a coefficient of only **+0.25** — i.e. it wants the clone flattened by a factor of four. Popularity is real signal about which move gets clicked, but as a distribution it is far too confident. That is a finding about the clone, not about this fit, and it is why the clone's own top-1 of 27.1% here sits below the 35.9% `eval_policy.py` reports: that harness scores over a move list, this one over (move, target) pairs, and the extra choice is the one the clone has no opinion about at all.

THE ASSUMPTION, WHICH IS THE INTERESTING PART: independence of irrelevant alternatives. Logit implies the odds between any two options do not depend on what else is available. That is known to fail when two alternatives are close substitutes — the red-bus/blue-bus problem — and Pokemon is full of close substitutes. A set carrying two Fire moves splits its "click a Fire move" mass across both, and logit will mis-state the odds against the third, dissimilar option. Nested or mixed logit is the standard remedy and neither is implemented.

So the fitted probabilities should be read as a good ranking and an **approximate** distribution, and the place it is most likely to be wrong is a set with redundant coverage. This is not a reason to distrust the realism numbers — those are measured from played games, not predicted — but it is a reason not to quote the per-decision probabilities as calibrated.

THE CORPUS OBJECTION, AND WHY IT SPLITS IN TWO. The fit uses open-team-sheet games, and the obvious objection is that open sheets change the incentives — a surprise set or a bluff item is worth nothing against someone who read your sheet before game one, so the **teams** are built differently, not merely

played differently. The corpus ships with a warning saying exactly that: *"Different information AND incentive regime … Do not pool."*

The objection is correct about teams and the effect is large. Measured by `engine/corpus_shift.js` over 2,136 clean open-sheet and 1,802 clean closed-sheet games:

	open-sheet	closed ladder
Garchomp on a team	81.6%	47.7%
Basculegion	61.3%	33.2%
Staraptor	44.0%	25.1%
Tyranitar	7.4%	21.2%
Sitrus Berry (share of items)	8.4%	17.5%

551.9 points of total absolute difference across 109 species. These are not the same metagame.

But the behaviour *given a board* — the thing the policy actually learns — is nearly identical:

	open-sheet	closed ladder	gap
super effective (of damaging moves)	35.59%	37.08%	1.49
resisted	15.06%	15.04%	0.02
immune	1.00%	0.97%	0.03
a move that could not work	1.30%	1.53%	0.23
status moves	33.89%	34.09%	0.20
Protect-family	13.87%	13.79%	0.08

That split is what licenses the corpus. The model is **conditional** — `P(choice | board, choice set)` — and it never learns what to bring; MEW samples its teams from the clean ladder store regardless. So the composition gap changes *which situations were sampled*, not *what was learned from them*.

That is still covariate shift, so it is corrected rather than argued away: `fit_policy.js` re-estimates on a sample importance-weighted to the closed-sheet species mix on **every run**.

AND THE WEIGHTS DO MOVE. An earlier version of this section said they did not, and it was wrong. That claim came from comparing the shift against a **hand-typed 0.25** — the invented constant S12/S13 exist to forbid. Conditional logit has the observed information in closed form, so every weight now carries a standard error and the shift is judged against the same z = 1.96 the project uses for every Wilson interval. Measured properly:

feature	open-sheet fit	reweighted to closed	shift
`priorLogP`	+0.241	+0.192	**10.8 SE**
`bp`	−0.131	+0.032	6.2 SE — *the sign flips*
`deadSide`	−2.293	−1.928	3.4 SE
`stab`	+0.164	+0.121	2.5 SE
`deadField`	−2.185	−1.883	2.1 SE

The objection therefore has real teeth, and lands exactly where it should: what moves is the **popularity** term and the **base power** term, not board-reading. `eff` (+0.800), `immune` (−2.073), `deadStatus` and `deadStall` are unmoved. Reading the board transfers between the two metagames; how much popularity is worth does not — unsurprising, since `priorLogP` is itself derived from the closed ladder.

The **reweighted vector ships**, because MEW samples its teams from the clean ladder store and the bot therefore plays in the closed-sheet metagame. Both vectors, the standard errors, and which one shipped are recorded in `data/policy-weights.json`.

The lesson generalises past this section and is the same one §2 records: a threshold that is *typed* rather than *derived* will eventually return the answer you assumed. Here it hid a 10-SE effect.

Three residual limits, stated rather than buried.

1. **Rating.** Open-sheet players average **1096** against the closed store's **1281**. Measured against the protocol on the closed store, though, move quality is close to flat in rating — failed moves run 2.59% at under 1100 and 2.30% at 1400–1600, and low-rated players hit *super effectively slightly more often*. Which is itself worth stating: **these realism metrics are not skill metrics.** Matching a human failure rate makes the bot human-like, not good, and above some point ALAKAZAM needs the second thing.
2. **Rule-ignorant plays exist and are not modelled.** Clicking a priority move into an ability that blocks it — Prankster Taunt into Farigiraf's Armor Tail, Fake Out into Queenly Majesty — appears 61 times in the clean closed store (Armor Tail 52, Queenly Majesty 9), and **no feature in `board.js` can represent it**: `immune` is computed from TYPES only. Ability-based immunity generally (Levitate, Flash Fire, Storm Drain, Sap Sipper) is the same hole and is the bigger half of it. Blocked-action rate does fall with rating, 4.66% under 1100 to 3.43% above 1400.
3. **Clicks that could not be matched to a candidate were dropped — and this document named the wrong cause for months.** It said the largest single cause was redirection (Follow Me, Rage Powder). **Measured 2026-08-02, `engine/redirect_audit.js`, 7,454 games: redirection is 1.60% of it.** The claim had never been measured; it was repeated across three documents and the fitter's own comments until it read as established.

Redirection *cannot* make a click unmatchable. The protocol emits no `-activate` line for a redirect and prints only the move's **resolved** target, so the redirector is a perfectly legal candidate: the matcher finds it and accepts the click — with the wrong label. That is a real defect and a small one, **1.55% of all clicks**, and it is unrecoverable from the store, because the chosen target was never written down by anyone at any point.

The real causes, by share of failures: the foe **switched in on the same turn** 44.4% — a human aims at a SLOT, the store records a SPECIES, and switches resolve before moves; an **in-battle forme change** with no sheet entry 19.7%; **no target recorded at all** 13.7%; and a **mirror collapsing the two team sheets** 16.4%. All four are addressed in `engine/click_match.js`, which moved the slot-level match rate from **87.2% to 97.2%** on the same replays scored twice.

- **Every `build_lab` win rate ON RECORD is still conditioned on a pilot that does not read the board**, and none has been re-run since §6's policy shipped. The pilot is no longer the only option — the scoring bot closed roughly half of each gap (super-effective 9.7% → 14.9% against a human 21.4%; failed moves 9.7% → 6.3% against 2.5%) — but no statistical correction repairs an already-published number, so every result in the lab remains provisional until it is re-measured.
- **The species under test is still piloted board-blind, deliberately.** `build_lab` needs equal airtime per arm (§4), so the scoring bot keeps the uniform pilot for exactly that species. The opponents now read the board and the tested Pokemon does not. This means §4's flaw — setup moves are not exchangeable in time, so setup builds are understated — **survives unchanged**, and an earlier note in the backlog claiming the scoring bot dissolved it was wrong.
- **The bot is one ply.** No damage calculation, no model of what the opponent will do, no search, and the switch decision is still `RandomPlayerAI`'s — 8.38 switches per game against a real 10.67.
- **External validity is untested.** `build_lab` measures one host team against 40 opponents and does not currently state that scope in its output.
- **24 engine tools** still read the ladder store without a clean filter or a declared reason. Their published numbers are unverified. `engine/selftest.js` fails on this by design.